TP2 – Rotas na Mina: Utilizando os Paradigmas Guloso e Divisão e Conquista

Thaylor Weslei Dias G. Verteiro
Matrícula: 2023028668
Universidade Federal de Minas Gerais (UFMG) – DCC

Novembro de 2026

1 Introdução e Modelagem do Problema

Neste trabalho, desenvolvi duas soluções para o problema de encontrar caminhos de menor custo em uma mina representada por uma grade bidimensional. Essa grade possui poucas linhas e um número maior de colunas, com dimensões $H \times W$, em que $2 \leq H \leq 5$ e $2 \leq W \leq 10.000$. Cada célula (i, j) possui um custo de travessia não negativo $c_{i,j}$, com valores entre 0 e 10^6.

Para representar o mapa, utilizei um grafo ponderado $G = (V, E)$:

- **Vértices (V):** Cada célula (r, c) representa um vértice do grafo, que pode ser identificado pelo índice $id = r \cdot W + c$. Dessa forma, o grafo possui $|V| = H \cdot W$ vértices.
- **Arestas (E):** Cada vértice pode se conectar às células vizinhas nas quatro direções: norte, sul, leste e oeste. Assim, cada vértice possui no máximo quatro vizinhos, e o número de arestas é limitado por $4 \cdot H \cdot W$.
- **Custo do caminho:** O custo de um percurso é calculado pela soma dos pesos de todas as células visitadas, incluindo a origem e o destino. Quando os dois pontos da consulta são a mesma célula, o custo é igual ao peso dessa célula.

O problema exige responder a um conjunto de Q consultas independentes, sendo $1 \leq Q \leq 50.000$. Para resolver essas consultas, implementei duas abordagens: uma utilizando o algoritmo de Dijkstra, dentro do paradigma guloso, e outra utilizando Divisão e Conquista.

2 Abordagem Gulosa (Dijkstra Direto)

Na primeira solução, utilizei o algoritmo de Dijkstra com uma fila de prioridades (*min-heap*). Nesse método, o algoritmo é executado separadamente para cada uma das Q consultas.

Para responder a uma consulta entre as células (r_1, c_1) e (r_2, c_2), começo atribuindo à origem uma distância igual ao seu próprio peso:

$$dist[start] = mapa[r_1][c_1]$$

Em seguida, os vértices são explorados de acordo com o menor custo acumulado. Quando o destino é retirado da fila de prioridades com sua menor distância definitiva, a busca pode ser encerrada, pois não é necessário continuar explorando os demais vértices.

2.1 Complexidade Teórica

- **Complexidade de Tempo:** Para cada consulta, o algoritmo de Dijkstra pode percorrer todo o grafo. Considerando $|V| = H \cdot W$ e o uso de uma fila de prioridades, sua complexidade é $O(|V| \log |V|)$. Como cada consulta é processada separadamente, a complexidade total é:

$$O(Q \cdot H \cdot W \log(H \cdot W))$$

- **Complexidade de Espaço:** São armazenados o mapa e o vetor de distâncias. Portanto, o espaço utilizado é $O(H \cdot W)$, sem considerar o armazenamento das consultas.

3 Abordagem por Divisão e Conquista

Na segunda solução, aproveitei o formato da grade, que possui poucas linhas e muitas colunas. A ideia é dividir o mapa em intervalos menores de colunas e resolver as consultas de forma recursiva.

Considerando um intervalo de colunas $[L, R]$, escolho sua coluna central:

$$m = \left\lfloor \frac{L + R}{2} \right\rfloor$$

A partir de cada uma das H células dessa coluna, executo Dijkstra, restringindo a busca às colunas do intervalo atual $[L, R]$.

Com as distâncias calculadas, consigo encontrar o menor custo de um caminho que passa por uma célula (k, m) da coluna central. Para isso, utilizo:

$$\text{custo_ponte} = \text{dist}_m[k][u_1] + \text{dist}_m[k][u_2] - c_{k,m} \tag{1}$$

Nessa expressão, $\text{dist}_m[k][u]$ representa a distância entre a célula (k, m) e o vértice u. O peso $c_{k,m}$ é subtraído porque ele aparece nas duas distâncias e, portanto, seria contado duas vezes.

Para cada consulta que está sendo processada no intervalo atual, testo as H células da coluna central e guardo o menor custo encontrado. Depois disso, as consultas cujas duas extremidades estão estritamente à esquerda ou à direita da coluna central são encaminhadas para as respectivas chamadas recursivas.

Esse processo permite considerar caminhos que passam pela coluna central e também caminhos que podem ser encontrados nas subdivisões seguintes. Assim, a resposta de cada consulta é atualizada com o menor custo encontrado durante a recursão.

3.1 Complexidade Teórica

- **Complexidade de Tempo:** A divisão do intervalo de colunas gera uma árvore de recursão com profundidade $O(\log W)$.

  Em cada intervalo, são executados H Dijkstras. Como cada busca trabalha com uma subgrade de largura s, sua complexidade é $O(Hs \log(Hs))$. Considerando as H buscas, o custo fica em $O(H^2 s \log(Hs))$.

  Como a soma das larguras dos intervalos de um mesmo nível é proporcional a W, o custo das buscas em cada nível é $O(H^2 W \log(HW))$. Como existem $O(\log W)$ níveis, o custo total das buscas é:

$$O(H^2 W \log(HW) \log W)$$

  Além disso, cada consulta é processada ao longo de, no máximo, $O(\log W)$ níveis, testando as H possibilidades da coluna central. Como H é limitado a 5, esse custo pode ser representado por $O(Q \log W)$.

Portanto, a complexidade total é:

$$O(H^2 W \log(HW) \log W + Q \log W)$$

- **Complexidade de Espaço:** São armazenados o mapa, as consultas e as respostas. Também são utilizadas estruturas auxiliares durante a recursão, como os vetores de distâncias e as listas de consultas dos subproblemas. Considerando essas estruturas, o espaço pode chegar a $O(H^2 W + Q \log W)$.

4 Resultados Experimentais

Para comparar as duas soluções, realizei os testes com as implementações compiladas utilizando a opção de otimização `-O2`. Utilizei os 31 casos de teste oficiais disponibilizados na disciplina.

Os tempos de execução foram medidos em segundos, com os resultados registrados com quatro casas decimais. A Figura 1 apresenta a comparação entre as duas abordagens para os 31 casos, utilizando escala logarítmica.

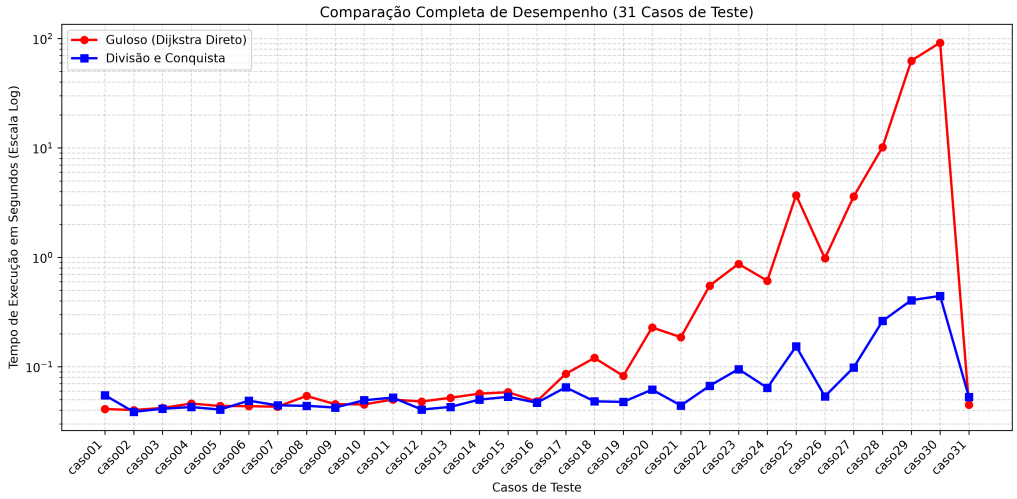


Figura 1: Comparação dos tempos de execução das duas abordagens nos 31 casos de teste, em escala logarítmica.

A Tabela 1 apresenta alguns dos resultados obtidos, mostrando como os tempos variam conforme aumentam a largura do mapa (W) e a quantidade de consultas (Q), mantendo $H = 5$.

5 Discussão dos Resultados

Observando os resultados, percebi que as duas abordagens apresentam tempos próximos nos casos menores. Nos casos iniciais, em que a quantidade de colunas e consultas é reduzida, a diferença entre os algoritmos é pequena. Isso acontece porque, nesses cenários, o custo das chamadas recursivas e das buscas adicionais da Divisão e Conquista pode se aproximar do custo de executar Dijkstras individuais.

A diferença fica mais evidente nos casos maiores. Conforme aumentam a largura do mapa e o número de consultas, o algoritmo Guloso precisa executar novas buscas de Dijkstra para cada consulta, repetindo cálculos que poderiam ser aproveitados.

Um exemplo disso é o caso 30, com $W = 10.000$ e $Q = 50.000$. Nesse teste, o algoritmo Guloso levou aproximadamente 91,63 segundos para terminar, enquanto a solução por Divisão e Conquista levou cerca de 0,44 segundo.

Tabela 1: Comparação dos tempos de execução em segundos para diferentes valores de W e Q, com $H = 5$.

Caso	**Colunas (W)**	**Consultas (Q)**	**Guloso (s)**	**Divisão e Conquista (s)**
caso01	2	1	0,0411	0,0550
caso03	5	10	0,0420	0,0413
caso05	10	20	0,0438	0,0406
caso08	20	40	0,0541	0,0439
caso10	30	60	0,0453	0,0494
caso13	100	200	0,0520	0,0429
caso15	200	300	0,0584	0,0531
caso17	400	500	0,0860	0,0648
caso20	1.000	1.000	0,2287	0,0618
caso23	2.500	2.500	0,8707	0,0951
caso25	4.000	4.000	3,6964	0,1534
caso28	8.000	10.000	10,1544	0,2617
caso29	10.000	25.000	62,7070	0,4059
caso30	10.000	50.000	91,6369	0,4438

Isso representa uma diferença de mais de 200 vezes no tempo de execução. Com esses resultados, pude observar que a Divisão e Conquista consegue lidar melhor com os casos de maior tamanho, principalmente porque organiza as buscas e aproveita a estrutura da grade para responder a várias consultas.

6 Conclusão

Com a realização dos testes, consegui observar as diferenças entre as duas abordagens utilizadas no trabalho.

A solução com Dijkstra direto é mais simples de implementar e apresenta resultados satisfatórios para entradas pequenas. Porém, seu desempenho diminui bastante quando o número de consultas e o tamanho do mapa aumentam, já que cada consulta exige uma nova execução do algoritmo.

Já a solução por Divisão e Conquista apresentou tempos menores nos casos maiores. Isso acontece porque ela utiliza a divisão do mapa em intervalos de colunas e organiza o processamento das consultas por meio de uma estrutura recursiva.

Dessa forma, considerando os resultados que obtive e as complexidades analisadas, concluo que a abordagem por Divisão e Conquista apresentou o melhor desempenho para os limites de entrada do problema.

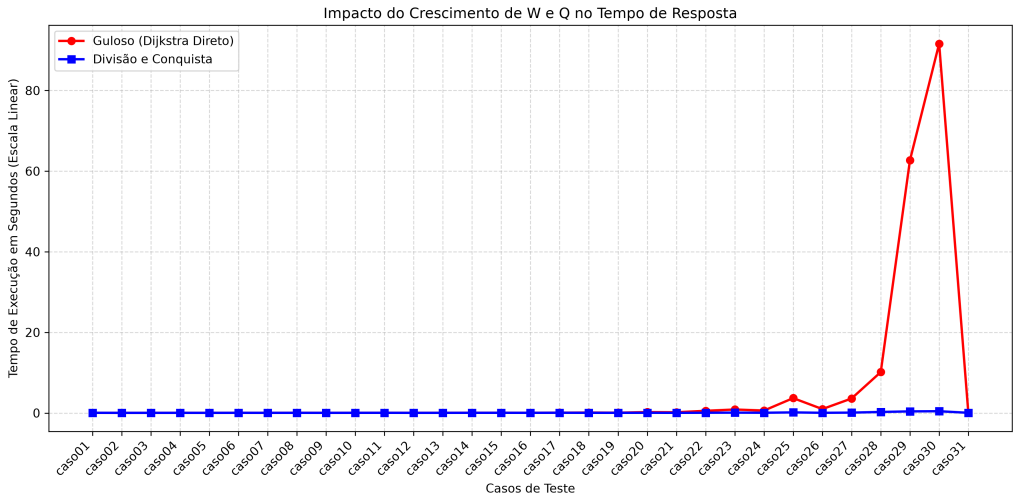


Figura 2: Comparação dos tempos de execução em escala linear, mostrando o crescimento do tempo conforme aumentam W e Q.